C++ COMPLETE REFERENCE LIBRARY • MODULE 01

# 01 CPP FUNDAMENTALS — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

## Module Table of Contents


1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 compilation pipeline and program structure
3. **Theory Chapter:** 02 iostream and namespace std
4. **Theory Chapter:** 03 outdated classroom vs modern cpp23
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions


# 01 — C++ Fundamentals

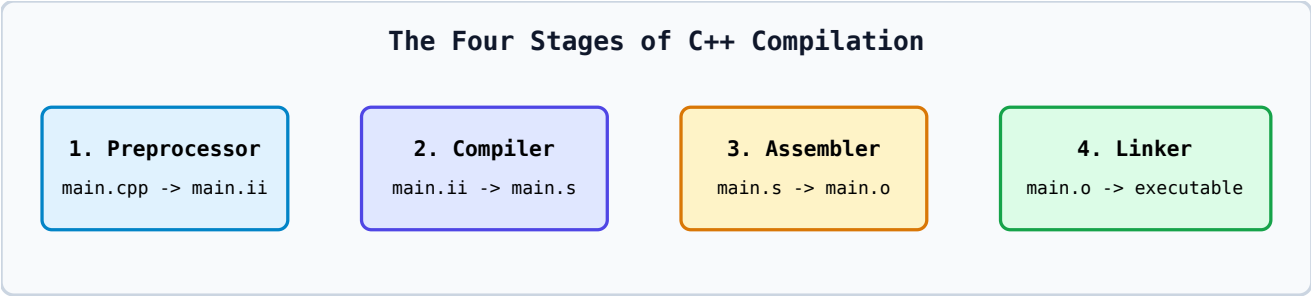
This module covers the C++ compilation pipeline (Preprocessing -> Compilation -> Assembly -> Linking), the structure of a standard C++23 program, `<iostream>` (`cin`, `cout`, `endl`), `int main()` vs. the non-standard `void main()`, and `using namespace std;` vs. explicit `std::` qualification.

## Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_compilation_pipeline_and_program_structure.md` (./01_THEORY/01_compilation_pipeline_and_program_structure.md)
  - `02_iostream_and_namespace_std.md` (./01_THEORY/02_iostream_and_namespace_std.md)
  - `03_outdated_classroom_vs_modern_cpp23.md` (./01_THEORY/03_outdated_classroom_vs_modern_cpp23.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_first_classroom_program.cpp` (./02_PROGRAMS/01_first_classroom_program.cpp) (Category A)
  - `02_basic_input_output_math.cpp` (./02_PROGRAMS/02_basic_input_output_math.cpp) (Category A)

  - `03_void_main_compile_error.cpp` (./02_PROGRAMS/03_void_main_compile_error.cpp) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_compilation_pipeline.svg` (./05_DIAGRAMS/diagram_01_compilation_pipeline.svg)

# 01 — The Four Stages of C++ Compilation and Program Structure




C++ Compilation Pipeline

## 1. The Four Stages of Turning `.cpp` into an Executable

1. **Preprocessing ( `g++ -E` )**: Expands `#include <iostream>` , `#define` macros, and strips comments, producing a translation unit ( `.i` / `.ii` ).
2. **Compilation ( `g++ -S` )**: Parses C++ syntax, checks types and access rules, optimizes, and generates architecture-specific assembly code ( `.s` ).
3. **Assembly ( `g++ -c` )**: Translates assembly instructions into machine-code object files ( `.o` / `.obj` ).
4. **Linking ( `ld` )**: Resolves external symbols across object files and standard library binaries ( `libstdc++` ) to produce the final executable binary.

## 2. Why `int main()` is Mandatory in Standard C++ ( `[basic.start.main]` )

The ISO C++ standard mandates that the return type of `main` in a hosted environment **must be** `int` .

- Writing `void main()` was a non-standard extension in 1990s MS-DOS compilers (like Turbo C++) and is **ill-formed (Compilation Error)** in modern standard C++ ( `g++ -pedantic-errors` ).
- If control reaches the end of `int main()` without encountering a `return` statement, the compiler implicitly inserts `return 0;` (indicating successful execution to the operating system).

# 02 — Input/Output Streams (`cin`, `cout`, `endl`) and `namespace std`

## 1. `<iostream>` Stream Objects

- `std::cout`: Standard character output stream (buffered), paired with the stream insertion operator `<<`.
- `std::cin`: Standard character input stream, paired with the stream extraction operator `>>` (skips leading whitespace).
- `std::endl` vs `'\n'`: `endl` outputs a newline `'\n'` **and flushes** the output buffer (`std::flush`). In performance-critical loops, `'\n'` is faster; in classroom programs, `endl` is convenient and clear.

## 2. `using namespace std;` vs. `std::` Qualification

- In simple single-file classroom programs, `using namespace std;` keeps code concise for written exams.
- In multi-file professional projects or header files (`.h` / `.hpp`), **never** write `using namespace std;` at global scope because it pulls thousands of identifiers (`count`, `distance`, `max`, `swap`) into the global namespace and causes name collisions!

# 03 — Converting Outdated Classroom C++ into Modern C++23

| Outdated / Legacy Classroom Syntax | Modern C++23 Standard Syntax | Reason for Change |
|---|---|---|
| `#include <iostream.h>` | `#include <iostream>` | Pre-1998 header removed from ISO C++ |
| `#include <conio.h>` / `clrscr()` / `getch()` | Omit (not needed in modern terminals) | Non-standard MS-DOS specific header |
| `void main()` | `int main()` | ISO C++ `[basic.start.main]` requires `int` |
| `int* p = NULL;` | `int* p = nullptr;` | Type-safe `std::nullptr_t` keyword |
| `char name[20]; gets(name);` | `std::string name; std::getline(cin, name);` | `gets()` was removed in C++14 due to buffer overflow vulnerabilities |

# Visual Concept & Memory Diagrams (01 CPP FUNDAMENTALS)

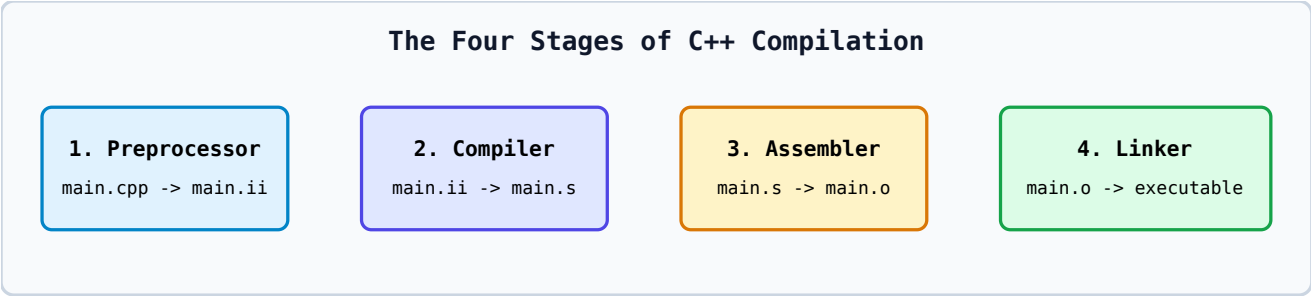



Diagram: diagram 01 compilation pipeline

# Annotated C++ Programs (01 CPP FUNDAMENTALS)

## Program: `01_first_classroom_program.cpp`

C++ SOURCE FILE • 01_first_classroom_program.cpp

```cpp
// Program Title: Standard Structure of a C++23 Classroom Program
// Topic: C++ Fundamentals
// Purpose: Demonstrate #include <iostream>, int main(), and console output.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_compilation_pipeline_and_program_structure.md
// Program Classification: Category A – Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    cout << "Welcome to B.Tech Mathematics and Computing C++23 Reference Library!" << endl;
    return 0;
}

/*
Expected Deterministic Output:
Welcome to B.Tech Mathematics and Computing C++23 Reference Library!
*/
```

## Program: `02_basic_input_output_math.cpp`

**C++ SOURCE FILE • 02_basic_input_output_math.cpp**

```cpp
// Program Title: Basic Arithmetic and Stream Chaining
// Topic: C++ Fundamentals — I/O Streams
// Purpose: Demonstrate variable output and chained << insertion operators.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_iostream_and_namespace_std.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

int main() {
    int a = 15;
    int b = 25;
    int sum = a + b;

    cout << "a = " << a << ", b = " << b << ", sum = " << sum << endl;
    return 0;
}

/*
Expected Deterministic Output:
a = 15, b = 25, sum = 40
*/
```

## Program: `03_void_main_compile_error.cpp`

**C++ SOURCE FILE • 03_void_main_compile_error.cpp**

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Non-Standard 'void main()' Rejection in Standard C++
// Topic: C++ Fundamentals — main() Return Type Rule
// Purpose: Show why 'void main()' is rejected by a conforming C++ compiler.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/01_compilation_pipeline_and_program_structure.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

void main() { // COMPILE ERROR: '::main' must return 'int'
    cout << "Hello" << endl;
}

/*
Why Compilation Fails:
ISO C++ [basic.start.main] requires the global main function to have a return type of 'int'.
Replace 'void main()' with 'int main()' as shown in 01_first_classroom_program.cpp.
*/
```

# Conceptual Questions — Module 01: C++ Fundamentals

**Q1.** Explain the 4 stages of translating a `.cpp` source file into an executable binary. **Q2.** Why is `void main()` invalid in standard C++, and what value does `int main()` return if `return 0;` is omitted?

# Debugging Questions — Module 01

**Q4.** Rewrite a legacy Turbo C++ program using `<iostream.h>`, `<conio.h>`, `void main()`, and `clrscr()` into valid C++23.

# Predict the Output Questions — Module 01

**Q3.** What is the difference between `cout << "A\n";` and `cout << "A" << endl;`?

# Programming Questions — Module 01

**Q5.** Write a C++23 program that computes the area and perimeter of a rectangle with given integer length and breadth.

# Detailed Solutions — Conceptual Questions (Module 01)

- **Q1 Solution**: (1) Preprocessing expands `#include` and macros; (2) Compilation checks syntax/types and produces assembly; (3) Assembly produces machine object code ( `.o` ); (4) Linking combines object files and libraries into an executable.
- **Q2 Solution**: ISO C++ `[basic.start.main]` mandates `int main()` so the program returns an exit status code to the operating system. If `return` is omitted at the end of `main()`, `return 0;` is implicitly executed.

# Detailed Solutions — Predict the Output & Debugging (Module 01)

- **Q3 Solution**: Both output `A` followed by a newline character, but `endl` additionally flushes the output stream buffer (`cout.flush()`).
- **Q4 Solution**: Replace `<iostream.h>` with `<iostream>`, remove `<conio.h>` and `clrscr()`, add `using namespace std;`, and change `void main()` to `int main()`.

# Detailed Solutions — Programming Questions (Module 01)

CPP

```cpp
#include <iostream>
using namespace std;
int main() {
    int length = 10, breadth = 5;
    cout << "Area: " << (length * breadth) << ", Perimeter: " << 2 * (length + breadth) <<
endl;
    return 0;
}
```